

Лекция 8. Основы объектно-ориентированного программирования и финальный проект

1) Актуальность темы

Объектно-ориентированное программирование (ООП) – это способ организации кода, при котором данные и действия над ними объединяются в классы. Почти все крупные библиотеки и фреймворки построены на ООП. Понимание классов открывает дорогу к грамотному использованию Django, pandas, tkinter и многих других инструментов.

2) Класс и объект

Класс – это пользовательский тип данных, описывающий состояние (атрибуты) и поведение (методы). Объект – конкретный экземпляр класса. Определение простейшего класса:

python

```
class Dog:
```

```
    def __init__(self, name, breed):
```

```
        self.name = name
```

```
        self.breed = breed
```

```
    def bark(self):
```

```
        print(f'{self.name} лает!')
```

Метод `__init__` – конструктор, вызывается при создании объекта. Параметр `self` ссылается на конкретный экземпляр и позволяет обращаться к его атрибутам.

Создание объектов:

python

```
my_dog = Dog("Бобик", "дворняга")
```

```
print(my_dog.name) # "Бобик"
```

```
my_dog.bark()      # "Бобик лает!"
```

3) Наследование

Наследование позволяет создать новый класс на основе существующего, перенеся его атрибуты и методы, и при необходимости переопределить их:

python

```
class GuardDog(Dog):
```

```
    def __init__(self, name, breed):
```

```
        super().__init__(name, breed) # вызов конструктора родителя
```

```
        self.duty = True
```

```
    def bark(self):
```

```
        print(f'{self.name} громко лает! Охраняет.')
```

Теперь GuardDog обладает всеми возможностями Dog, но метод bark работает иначе.

4) Инкапсуляция

В Python нет жёсткого сокрытия, но принято соглашение: атрибуты и методы, начинающиеся с одного подчёркивания (`_protected`), считаются внутренними и не предназначены для использования извне. Два подчёркивания (`__private`) вызывают искажение имени, усложняя случайный доступ.

5) Финальный проект: система управления задачами

Объединим все знания в мини-проекте. Создадим класс Task и менеджер задач с возможностью добавлять, удалять и отмечать выполнение.

python

```
class Task:
```

```
    def __init__(self, title):
```

```
        self.title = title
```

```
        self.done = False
```

```
    def complete(self):
```

```
        self.done = True
```

```
    def __str__(self):
```

```
        status = "✓" if self.done else "X"
```

```
        return f"[{status}] {self.title}"
```

```
class TaskManager:
```

```
    def __init__(self):
```

```
        self.tasks = []
```

```
    def add(self, title):
```

```
        self.tasks.append(Task(title))
```

```
    def remove(self, index):
```

```
        if 0 <= index < len(self.tasks):
```

```
            del self.tasks[index]
```

```
    def show(self):
```

```
for i, task in enumerate(self.tasks):  
    print(f'{i}: {task}')
```

```
def mark_done(self, index):  
    if 0 <= index < len(self.tasks):  
        self.tasks[index].complete()
```

Использование:

```
python  
  
tm = TaskManager()  
tm.add("Выучить Python")  
tm.add("Написать проект")  
tm.mark_done(0)  
tm.show()
```

Этот проект можно расширять: сохранять задачи в файл, фильтровать по статусу, добавлять сроки.

6) Куда двигаться дальше

После освоения базы Python открыты все направления:

- Веб-разработка – фреймворки Django/Flask
- Анализ данных – библиотеки pandas, matplotlib, jupyter
- Машинное обучение – scikit-learn, TensorFlow
- Автоматизация – написание скриптов, работа с API
- Игры – pygame

Важно продолжать практиковаться и постепенно углубляться в выбранную область, помня, что надёжный фундамент уже заложен.

Итоги лекции (что нужно запомнить)

- Класс – шаблон, объект – экземпляр; атрибуты хранят данные, методы описывают поведение.
- `__init__` – конструктор, `self` – ссылка на объект.
- Наследование позволяет расширять классы, не переписывая код.
- Инкапсуляция в Python на уровне соглашений (`_` и `__`).
- Финальный проект демонстрирует применение всех пройденных тем: классы, списки, условия, циклы, вывод.